CashPilot AI:

Project type: Agentic AI for SME Financial Management

Core concept: An AI financial operations agent that helps SMEs understand their financial position, anticipate cash shortages, and take action on financial problems.

1. Frontend Design (Dashboard + AI Chat + Telegram)

1.1 Product vision

Small and medium-sized enterprises often manage financial information through spreadsheets, invoices, bank statements, and messaging apps. Business owners may know their sales figures but lack a clear picture of upcoming cash shortages, overdue payments, and financial risks.

CashPilot AI addresses this problem through an AI agent that connects financial data, forecasting tools, a conversational interface, and controlled business actions.

The product will allow an owner to upload invoices, ask financial questions, receive proactive alerts, and instruct the agent to prepare follow-up actions.

1.2 Frontend:

We should build a **web-based financial dashboard with an integrated AI chat interface and a Telegram bot**.

The dashboard provides a financial overview, while the chat and Telegram interfaces let the owner interact with the same underlying agent.

Main dashboard

The dashboard should contain the following sections:

1. Financial overview

- Current cash balance.
- Expected incoming payments.
- Upcoming outgoing payments.
- Projected minimum cash balance.
- Overdue receivables.
- Active financial alerts.

2. Cash-flow forecast

A chart displaying projected cash balances over the next 30 days, including expected inflows, outflows, and dates when the balance may fall below a configured reserve.

3. Alerts and recommended actions

A prioritized list of issues, such as:

- A projected cash shortage in eight days.
- Three overdue customer invoices.
- A large supplier payment due tomorrow.
- An unusual expense requiring review.

Each alert should explain its cause and provide a suggested next action.

4. Invoice management

A searchable table containing invoice numbers, customers, amounts, due dates, payment status, and available follow-up actions.

5. AI copilot

A chat panel that supports questions such as:

- "Can I afford to purchase ৳200,000 of inventory?"
- "Which customers owe me the most money?"
- "Why is my projected cash balance falling?"
- "Prepare reminders for overdue invoices."

6. Activity and audit log (IMPORTANT FOR SECURITY)

A record of what the agent analyzed, recommended, prepared, and executed, including the relevant timestamps and approval status.

1.3 Telegram integration

Telegram will provide a second interface to the same agent.

Business owners will be able to ask questions about their finances/send invoice PDFs to the telegram bot: The agent extracts the invoice details, validates the data, saves the invoice, recalculates the financial forecast, and sends a confirmation. The updated invoice and its financial impact should also appear on the web dashboard.

Important architectural decision: Telegram and the web application must share the same backend, business records, financial calculation engine, and agent tools. They should not behave as two independent bots.

For the hackathon, invoice ingestion through Telegram can be implemented alongside the web dashboard's CSV upload. PDF and image extraction can use OCR or a document-understanding model.

2. Agentic Architecture

2.1 Core architecture

The system will consist of these components:

Frontend and communication channels

- Web dashboard and chat interface.
- Telegram bot.
- Shared authentication and user identity.

Backend API

- Handles requests from both interfaces.
- Manages business records and file uploads.
- Enforces authentication, permissions, and approval requirements.

Agent orchestration layer

- Uses an LLM with tool calling.
- Determines which financial tools to invoke.
- Interprets tool results and coordinates multi-step workflows.

Financial tools and calculation engine

- Transaction and invoice lookup.

- Cash-flow forecasting.
- Overdue invoice detection.
- Financial alert generation.
- Invoice reminder drafting.
- Scenario analysis.

Data layer

- Business profiles.
- Invoices and transactions.
- Payment schedules and financial settings.
- Alerts, recommendations, and action logs.
- Uploaded documents and their extraction status.

External integrations

- Telegram Bot API.
- LLM API.
- OCR or document-understanding service.
- Optional sandbox banking integration, if available and authorized.

2.2 Tool design

The following tools should be implemented as explicit backend functions:

Tool	Purpose
`get_financial_summary`	Retrieve current financial metrics
`get_invoices`	Search invoices by customer, amount, or status
`get_upcoming_payables`	Retrieve scheduled business obligations
`get_upcoming_receivables`	Retrieve expected customer payments
`forecast_cash_balance`	Calculate future balances
`get_financial_alerts`	Retrieve and prioritize detected risks
`create_reminder_draft`	Prepare an invoice follow-up
`record_action_status`	Record the status of an approved workflow

The LLM chooses and coordinates tools, but financial arithmetic and permission checks remain in deterministic backend code.

3. Feature 1 — AI Financial Data Analyzer

Short description: A data-ingestion and validation system that converts a business's raw financial information into structured records the copilot can use.

Problem: SMEs often maintain financial data across spreadsheets, invoices, receipts, and transaction exports. Manual entry and inconsistent records make financial analysis time-consuming and error-prone.

Implementation

Step 1: Support financial data uploads

Implement CSV uploads through the web dashboard. Accept a predefined schema containing:

- Transaction date.
- Description.

- Amount.
- Transaction type.
- Customer or supplier.
- Invoice number, where available.
- Due date and payment status, where available.

Use a separate invoice schema for accounts receivable and accounts payable.

Step 2: Validate and normalize data

Use backend validation to:

- Parse dates and currency amounts.
- Normalize transaction types.
- Detect missing required fields.
- Identify likely duplicate records.
- Flag inconsistent invoice totals and payment statuses.

Do not silently discard invalid records. Return a report explaining which rows require correction.

Step 3: Extract invoice information

For PDF or image invoices, use OCR or a document-understanding model to extract:

- Invoice number.
- Supplier or customer.
- Issue date.
- Due date.
- Currency and total amount.
- Relevant tax or payment information.

Display extracted information for confirmation before saving uncertain records.

Step 4: Store the records

Save validated records in the application's database and associate them with the correct business account.

Step 5: Recalculate financial metrics

Trigger a refresh of the financial summary, overdue invoice list, and cash-flow forecast when new records are accepted.

4. Feature 2 — Cash-Flow Forecasting Engine

Short description: A deterministic forecasting engine that estimates how much cash the business will have on each of the next 30 days.

Problem: A business can have strong sales and still be unable to pay suppliers because customer payments arrive later than expenses become due.

Implementation

Step 1: Establish the opening balance

Use the latest known cash balance from the sample dataset or a user-entered opening balance.

Step 2: Identify future cash movements

Collect:

- Expected customer receipts.
- Scheduled supplier payments.
- Rent and payroll.
- Loan repayments.
- Other known business expenses.

Step 3: Calculate the daily forecast

Use the following relationship:

Projected closing cash balance = Opening cash balance + Expected inflows − Expected outflows.

For each day, the previous day's closing balance becomes the next day's opening balance.

Step 4: Handle uncertainty

Separate confirmed payments from expected receipts and uncertain estimates. Allow the user to define a conservative assumption for late customer payments.

For the MVP, use a simple scenario-based model rather than training a predictive machine-learning model.

Step 5: Detect low-cash periods

Compare the projected balance with a user-configured minimum cash reserve.

Flag dates where:

- The projected balance falls below the reserve.
- The projected balance becomes negative.
- A major payment creates a sharp reduction in available cash.

Step 6: Visualize the forecast

Display a 30-day chart, the minimum projected balance, the date of the lowest balance, and the transactions driving that result.

5. Feature 3 — Intelligent Financial Alerts

Short description:A rule-based alert engine that detects important financial risks and prioritizes them according to urgency and financial impact.

Problem: Owners may have all the necessary financial information but lack the time to inspect it regularly. They discover overdue invoices and cash shortages only when they become urgent.

Implementation

Step 1: Implement financial rules

For the MVP, create rules for:

- Cash falling below the configured reserve.
- Projected negative cash balance.
- Overdue customer invoices.
- Supplier payments due within a configurable number of days.

- Large or unusual expenses relative to a defined baseline.

Step 2: Calculate alert severity

Assign a severity level based on:

- Financial impact.
- Time remaining before the event.
- Whether the problem is confirmed or only projected.

Use deterministic rules rather than asking the LLM to invent severity scores.

Step 3: Explain the reason

Attach the supporting records to each alert.

For example, a cash-shortage alert should identify the expected inflows and outflows responsible for the projected deficit.

Step 4: Display alerts

Create a dashboard panel containing:

- Alert title.
- Severity.
- Relevant date.
- Estimated financial impact.
- Supporting transactions.
- Suggested next action.

Step 5: Connect alerts to the agent

Expose a `get_financial_alerts` tool so the agent can retrieve, explain, and prioritize the same alerts shown on the dashboard.

6. Feature 4 — Agentic Financial Copilot

Short description: A conversational financial agent that answers questions using company records and financial tools, investigates problems, and coordinates follow-up actions.

Problem: Traditional financial dashboards display metrics but require the owner to interpret them and decide what to do. A generic LLM chatbot, meanwhile, may provide advice without actually checking the business's financial records.

Implementation

Step 1: Create the chat interface

Add a chat panel to the dashboard. Support natural-language questions about invoices, cash flow, upcoming payments, and financial alerts.

Step 2: Implement the agent orchestration layer

Connect an LLM that supports tool calling to the backend financial tools.

Provide a system-level instruction explaining:

- The agent's role.
- Which tools are available.
- Which financial calculations must use backend tools.
- How to handle missing data.
- Which actions require user approval.

Step 3: Implement multi-step workflows

Allow the agent to:

1. Interpret the user's request.
2. Select the relevant tools.
3. Retrieve records and calculate results.
4. Evaluate the results against the user's question.
5. Call additional tools if required.
6. Produce an evidence-backed answer.
7. Offer or prepare an appropriate next action.

Step 4: Connect Telegram

Create a Telegram bot using the Telegram Bot API.

Route text messages and document uploads to the same backend agent used by the web chat. Associate each Telegram account with an authorized business user.

Implement document-size limits, file-type validation, authentication, and appropriate access controls.

Step 5: Add memory and context

For the MVP, store the user's business settings, conversation history, and current workflow state in the database. Retrieve only the relevant context for each request.

Avoid implementing a complex autonomous memory system during the sprint.

Step 6: Add action permissions

Distinguish between:

- Read-only operations.
- Internal operations such as saving a validated invoice.
- External actions such as sending a customer message.
- Financial transactions such as initiating a payment.

Require appropriate authorization before consequential actions. For the hackathon, actual money movement is unnecessary.

7. Feature 5 — Invoice Follow-up and Action Planner

Short description: An action-oriented workflow that prioritizes overdue receivables, drafts customer reminders, and tracks follow-up status.

Problem: Small businesses lose time chasing payments manually. Owners may forget to contact customers or fail to prioritize invoices that would make the biggest difference to cash flow.

Implementation

Step 1: Identify overdue invoices

Filter invoices by due date and outstanding balance. Exclude paid invoices and correctly account for partial payments.

Step 2: Prioritize follow-ups

Use a simple scoring rule based on:

- Days overdue.
- Outstanding amount.
- Urgency of the projected cash shortage.
- Customer payment history, if available.

For the MVP, use transparent rules rather than a trained prediction model.

Step 3: Generate a reminder draft

Use the LLM to produce a professional reminder containing:

- Customer name.
- Invoice reference.
- Outstanding balance.
- Original due date.
- A request for payment confirmation or an expected payment date.

Pass verified invoice data to the LLM. Do not allow it to invent payment details or contractual terms.

Step 4: Add an approval workflow:

Display the draft to the user, who can edit, copy, approve, or mark it as completed.

Sending a real message should require explicit approval unless the user has separately configured an authorized sending workflow.

Step 5: Track the outcome

Record the invoice, reminder status, timestamp, and any subsequent payment updates.

Update the outstanding balance when a payment is verified.